AI-Assisted Coding
Lab Assignment - 1 7.1

Name: A. Sushmitha
Roll No: 2303A51541
Batch No: 13

Task 1 – Customer Data Cleaning Task:
Use AI to generate a Python script for cleaning a customer dataset.

Sample Code:

```
import pandas as pd

data = pd.read_csv("shopping_trends.csv")

print(data.head())
```

Dataset link:

https://www.kaggle.com/datasets/iamsouravbanerjee/customer-shoppin
g- trends-dataset?select=shopping_trends.csv

Expected Output:

- A cleaned Pandas data frame ready for analysis.

Prompt:

Generate a Python pandas script to clean a customer dataset (shopping_trends.csv) by handling missing values (age, income, city), removing duplicates, standardizing city names to lowercase, encoding categorical variables (gender, city), and outputting a
cleaned dataframe.

Code:

```python
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Load dataset
data = pd.read_csv("shopping_trends.csv")

# Display original data
print("Original Data:")
print(data.head())

# 1. Handle Missing Values

# Fill missing age with median
if 'Age' in data.columns:
    data['Age'].fillna(data['Age'].median(), inplace=True)

# Fill missing income with median (if exists)
if 'Income' in data.columns:
    data['Income'].fillna(data['Income'].median(), inplace=True)

# Fill missing city with mode (most frequent)
if 'Location' in data.columns:
    data['Location'].fillna(data['Location'].mode()[0], inplace=True)


# 2. Remove Duplicates
```

```python
# 2. Remove Duplicates
data.drop_duplicates(inplace=True)

# 3. Standardize City Names to Lowercase
if 'Location' in data.columns:
    data['Location'] = data['Location'].str.lower().str.strip()
encoder = LabelEncoder()

# Encode Gender
if 'Gender' in data.columns:
    data['Gender'] = encoder.fit_transform(data['Gender'])

# Encode City (Location)
if 'Location' in data.columns:
    data['Location'] = encoder.fit_transform(data['Location'])
# Display cleaned data
print("\nCleaned Data:")
print(data.head())

# Save cleaned dataset (optional)
data.to_csv("cleaned_shopping_trends.csv", index=False)
```

Output: Original data

```
Original Data:
   Customer ID  Age Gender Item Purchased  Category  Purchase Amount (USD)  \
0            1   55   Male         Blouse  Clothing                     53
1            2   19   Male        Sweater  Clothing                     64
2            3   50   Male          Jeans  Clothing                     73
3            4   21   Male        Sandals  Footwear                     90
4            5   45   Male         Blouse  Clothing                     49

        Location Size      Color  Season  Review Rating Subscription Status  \
0       Kentucky    L       Gray  Winter            3.1                 Yes
1          Maine    L     Maroon  Winter            3.1                 Yes
2  Massachusetts    S     Maroon  Spring            3.1                 Yes
3   Rhode Island    M     Maroon  Spring            3.5                 Yes
4         Oregon    M  Turquoise  Spring            2.7                 Yes

  Payment Method  Shipping Type Discount Applied Promo Code Used  \
0    Credit Card        Express              Yes             Yes
1   Bank Transfer       Express              Yes             Yes
2           Cash  Free Shipping              Yes             Yes
3         PayPal   Next Day Air              Yes             Yes
4           Cash  Free Shipping              Yes             Yes

   Previous Purchases Preferred Payment Method Frequency of Purchases
0                  14                    Venmo            Fortnightly
1                   2                     Cash            Fortnightly
2                  23              Credit Card                 Weekly
3                  49                   PayPal                 Weekly
4                  31                   PayPal               Annually

Cleaned Data:
```

Cleaned data:

```
Cleaned Data:
   Customer ID  Age  Gender Item Purchased  Category  Purchase Amount (USD)  \
0            1   55       1         Blouse  Clothing                     53
1            2   19       1        Sweater  Clothing                     64
2            3   50       1          Jeans  Clothing                     73
3            4   21       1        Sandals  Footwear                     90
4            5   45       1         Blouse  Clothing                     49

   Location Size      Color  Season  Review Rating Subscription Status  \
0        16    L       Gray  Winter            3.1                 Yes
1        18    L     Maroon  Winter            3.1                 Yes
2        20    S     Maroon  Spring            3.1                 Yes
3        38    M     Maroon  Spring            3.5                 Yes
4        36    M  Turquoise  Spring            2.7                 Yes

  Payment Method  Shipping Type Discount Applied Promo Code Used  \
0    Credit Card        Express              Yes             Yes
1  Bank Transfer        Express              Yes             Yes
2           Cash  Free Shipping              Yes             Yes
3         PayPal   Next Day Air              Yes             Yes
4           Cash  Free Shipping              Yes             Yes

   Previous Purchases Preferred Payment Method Frequency of Purchases
0                  14                    Venmo            Fortnightly
1                   2                     Cash            Fortnightly
2                  23              Credit Card                 Weekly
3                  49                   PayPal                 Weekly
4                  31                   PayPal               Annually
/tmp/ipykernel_4119/4199330253.py:15: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the c
```

Justification:

This script performs the following steps:

1. Loads the dataset from 'shopping_trends.csv'.
2. Handles missing values by filling 'age' and 'income' with their median values and 'city' with 'unknown'.
3. Removes duplicate rows from the dataset.
4. Standardizes city names to lowercase.
5. Encodes the 'gender' column to numerical values and creates dummy variables for the 'city' column.
6. Outputs the cleaned dataframe and saves it to 'cleaned_shopping_trends.csv'.

Task 2 – Sales Data Preprocessing

Task:

Preprocess sales transaction data using AI assistance.

Sample Code:

import pandas as pd

data = pd.read_csv("sales_data.csv")

print(data.head())

Dataset link:

https://www.kaggle.com/datasets/vinothkannaece/sales-dataset?select=sales_data.csv

Expected Output:

- A transformed dataset with time-based features and normalized values.

Prompt:

Generate a Python pandas script to preprocess a sales dataset (sales_data.csv) by converting date columns to datetime, creating new features (month, year, day of week), normalizing the sales amount column, detecting and handling outliers, and outputting the transformed dataset.

Code:

```
[2]
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Load dataset
try:
    sales_data = pd.read_csv("sales_data.csv")
except FileNotFoundError:
    print("Error: sales_data.csv not found. Please ensure the file is in the correct directory.")
    exit()

print("Original Sales Data:")
print(sales_data.head())

# 1. Convert date columns to datetime
# Assuming 'OrderDate' is the date column. Adjust if necessary.
if 'OrderDate' in sales_data.columns:
    sales_data['OrderDate'] = pd.to_datetime(sales_data['OrderDate'], errors='coerce')
    # Drop rows where date conversion failed
    sales_data.dropna(subset=['OrderDate'], inplace=True)

# 2. Create new features (month, year, day of week)
if 'OrderDate' in sales_data.columns:
    sales_data['SaleMonth'] = sales_data['OrderDate'].dt.month
    sales_data['SaleYear'] = sales_data['OrderDate'].dt.year
    sales_data['SaleDayOfWeek'] = sales_data['OrderDate'].dt.dayofweek # Monday=0, Sunday=6
```

```
# 3. Normalize the sales amount column
# Assuming 'SalesAmount' is the sales amount column. Adjust if necessary.
if 'SalesAmount' in sales_data.columns:
    scaler = MinMaxScaler()
    sales_data['NormalizedSalesAmount'] = scaler.fit_transform(sales_data[['SalesAmount']])

# 4. Detect and handle outliers (using IQR method for 'SalesAmount')
if 'SalesAmount' in sales_data.columns:
    Q1 = sales_data['SalesAmount'].quantile(0.25)
    Q3 = sales_data['SalesAmount'].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Option 1: Cap outliers
    sales_data['SalesAmount_Capped'] = sales_data['SalesAmount'].clip(lower=lower_bound, upper=upper_bound)

    # Option 2: Remove outliers (uncomment if you prefer removal)
    # sales_data = sales_data[(sales_data['SalesAmount'] >= lower_bound) & (sales_data['SalesAmount'] <= upper_bound)]

print("\nTransformed Sales Data (with new features, normalized and capped sales amount):")
print(sales_data.head())

# Optional: Save the transformed dataset
sales_data.to_csv("transformed_sales_data.csv", index=False)
```

Output: Original Sales data

```
Original Sales Data:
   Product_ID   Sale_Date Sales_Rep Region  Sales_Amount  Quantity_Sold  \
0        1052  2023-02-03       Bob  North       5053.97             18
1        1093  2023-04-21       Bob   West       4384.02             17
2        1015  2023-09-21     David  South       4631.23             30
3        1072  2023-08-24       Bob  South       2167.94             39
4        1061  2023-03-24   Charlie   East       3750.20             13

  Product_Category  Unit_Cost  Unit_Price Customer_Type  Discount  \
0       Furniture     152.75      267.22     Returning      0.09
1       Furniture    3816.39     4209.44     Returning      0.11
2            Food     261.56      371.40     Returning      0.20
3        Clothing    4330.03     4467.75           New      0.02
4     Electronics     637.37      692.71           New      0.08

  Payment_Method Sales_Channel Region_and_Sales_Rep
0           Cash        Online            North-Bob
1           Cash        Retail             West-Bob
2  Bank Transfer        Retail          South-David
3    Credit Card        Retail            South-Bob
4    Credit Card        Online         East-Charlie
```

Transformed Sales data:

```
  Payment_Method Sales_Channel Region_and_Sales_Rep
0           Cash        Online            North-Bob
1           Cash        Retail             West-Bob
2  Bank Transfer        Retail          South-David
3    Credit Card        Retail            South-Bob
4    Credit Card        Online         East-Charlie

Transformed Sales Data (with new features, normalized and capped sales amount):
   Product_ID   Sale_Date Sales_Rep Region  Sales_Amount  Quantity_Sold  \
0        1052  2023-02-03       Bob  North       5053.97             18
1        1093  2023-04-21       Bob   West       4384.02             17
2        1015  2023-09-21     David  South       4631.23             30
3        1072  2023-08-24       Bob  South       2167.94             39
4        1061  2023-03-24   Charlie   East       3750.20             13

  Product_Category  Unit_Cost  Unit_Price Customer_Type  Discount  \
0       Furniture     152.75      267.22     Returning      0.09
1       Furniture    3816.39     4209.44     Returning      0.11
2            Food     261.56      371.40     Returning      0.20
3        Clothing    4330.03     4467.75           New      0.02
4     Electronics     637.37      692.71           New      0.08

  Payment_Method Sales_Channel Region_and_Sales_Rep
0           Cash        Online            North-Bob
1           Cash        Retail             West-Bob
2  Bank Transfer        Retail          South-David
3    Credit Card        Retail            South-Bob
4    Credit Card        Online         East-Charlie
```

Justification:

Sales data preprocessing improves data quality and ensures accurate analysis by converting dates and handling inconsistencies.
Feature engineering (month, year, day) helps identify trends and patterns over time. Normalization and outlier handling make the dataset more reliable for machine learning and statistical analysis.

Task 3 – Healthcare Data Preparation

Task:

Clean and preprocess a healthcare dataset using AI scripts.

Dataset link :

https://www.kaggle.com/datasets/abdallaahmed77/healthcare-risk-factors-dataset

Expected Output:

- A preprocessed dataset ready for machine learning models.

Prompt:

Generate a Python pandas script to preprocess a healthcare dataset by handling missing values (blood_pressure, cholesterol), encoding categorical features, scaling numerical features using min-max or standard scaling, splitting the dataset into training and testing sets, and outputting a dataset ready for machine learning.

Code:

Gemini

```python
import pandas as pd
from sklearn.preprocessing import LabelEncoder, MinMaxScaler, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer

# Load dataset
try:
    df = pd.read_csv("healthcare_dataset.csv")
except FileNotFoundError:
    print("Error: healthcare_dataset.csv not found. Please ensure the file is in the correct directory.")
    # Create a dummy DataFrame for demonstration if file not found
    data = {
        'Age': [30, 45, 50, 60, 25, 35, 55, 40, 65, 70],
        'Gender': ['Male', 'Female', 'Male', 'Female', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male'],
        'blood_pressure': [120, 140, None, 130, 110, 125, 150, None, 135, 160],
        'cholesterol': [200, 240, 220, None, 180, 210, 250, 190, None, 230],
        'HeartRate': [70, 85, 75, 90, 65, 80, 95, 72, 88, 78],
        'Smoker': ['Yes', 'No', 'No', 'Yes', 'No', 'Yes', 'No', 'No', 'Yes', 'No'],
        'Outcome': [0, 1, 0, 1, 0, 0, 1, 0, 1, 1]
    }
    df = pd.DataFrame(data)
    print("Using a dummy healthcare dataset for demonstration.")

print("Original Data Head:")
print(df.head())
print("\nMissing Values before preprocessing:")
print(df.isnull().sum())
```

```python
# 1. Handle Missing Values
# For 'blood_pressure' and 'cholesterol', fill with the median
if 'blood_pressure' in df.columns:
    df['blood_pressure'].fillna(df['blood_pressure'].median(), inplace=True)
if 'cholesterol' in df.columns:
    df['cholesterol'].fillna(df['cholesterol'].median(), inplace=True)

print("\nMissing Values after handling blood_pressure and cholesterol:")
print(df.isnull().sum())

# 2. Encode Categorical Features
categorical_cols = df.select_dtypes(include='object').columns
encoder = LabelEncoder()
for col in categorical_cols:
    df[col] = encoder.fit_transform(df[col])
    print(f"Encoded column '{col}' with LabelEncoder.")
# Note: For non-ordinal categorical features, consider using OneHotEncoder instead of LabelEncoder.
# Example: df = pd.get_dummies(df, columns=['Gender', 'Smoker'], drop_first=True)

# 3. Scale Numerical Features
# Identify numerical columns (excluding the target if it exists)
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
# Assuming 'Outcome' is the target variable, exclude it from scaling if present
if 'Outcome' in numerical_cols:
    numerical_cols.remove('Outcome')

scaler = MinMaxScaler() # Using Min-Max Scaling
# Alternative: scaler = StandardScaler() # Using Standard Scaling

df[numerical_cols] = scaler.fit_transform(df[numerical_cols])
```

```python
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])
print(f"\nScaled numerical columns: {numerical_cols} using MinMaxScaler.")

# 4. Split the dataset into training and testing sets
# Assuming 'Outcome' is the target variable. Adjust if your target column has a different name.
if 'Outcome' in df.columns:
    X = df.drop('Outcome', axis=1)
    y = df['Outcome']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
    print("\nDataset split into training and testing sets (80/20 split).")
    print(f"X_train shape: {X_train.shape}")
    print(f"X_test shape: {X_test.shape}")
    print(f"y_train shape: {y_train.shape}")
    print(f"y_test shape: {y_test.shape}")
else:
    print("Warning: 'Outcome' column not found for train-test split. Skipping split.")
    X = df
    print(f"Processed DataFrame (X) shape: {X.shape}")
    # If no target, you might just want to save the processed features

print("\nPreprocessed Data Head (ready for machine learning):")
print(df.head())

# Outputting the preprocessed dataset (optional)
df.to_csv("preprocessed_healthcare_dataset.csv", index=False)
print("\nPreprocessed data saved to 'preprocessed_healthcare_dataset.csv'")
```

Output: Original data before Preprocessing

```
Original Data Head:
   Age  Gender  blood_pressure  cholesterol  HeartRate Smoker  Outcome
0   30    Male           120.0        200.0         70    Yes        0
1   45  Female           140.0        240.0         85     No        1
2   50    Male             NaN        220.0         75     No        0
3   60  Female           130.0          NaN         90    Yes        1
4   25  Female           110.0        180.0         65     No        0

Missing Values before preprocessing:
Age                0
Gender             0
blood_pressure     2
cholesterol        2
HeartRate          0
Smoker             0
Outcome            0
dtype: int64
```

Data After Preprocessing

```
Missing Values after handling blood_pressure and cholesterol:
Age                0
Gender             0
blood_pressure     0
cholesterol        0
HeartRate          0
Smoker             0
Outcome            0
dtype: int64
Encoded column 'Gender' with LabelEncoder.
Encoded column 'Smoker' with LabelEncoder.

Scaled numerical columns: ['Age', 'Gender', 'blood_pressure', 'cholesterol', 'HeartRate', 'Smoker'] using MinMaxScaler.

Dataset split into training and testing sets (80/20 split).
X_train shape: (8, 6)
X_test shape: (2, 6)
y_train shape: (8,)
y_test shape: (2,)

Preprocessed Data Head (ready for machine learning):
        Age  Gender  blood_pressure  cholesterol  HeartRate  Smoker  Outcome
0  0.111111     1.0            0.20     0.285714   0.166667     1.0        0
1  0.444444     0.0            0.60     0.857143   0.666667     0.0        1
2  0.555556     1.0            0.45     0.571429   0.333333     0.0        0
3  0.777778     0.0            0.40     0.500000   0.833333     1.0        1
4  0.000000     0.0            0.00     0.000000   0.000000     0.0        0

Preprocessed data saved to 'preprocessed_healthcare_dataset.csv'
/tmp/ipykernel_4119/1202613681.py:32: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the c
```

Justification:

Preprocessing healthcare data ensures accuracy by handling missing values and inconsistencies in medical attributes.
Encoding and scaling features make the data suitable for machine learning algorithms.
Splitting the dataset helps evaluate model performance effectively on unseen data.

Task 4 – Employee Salary Data Preprocessing

Task:

Clean and preprocess an employee salary dataset using AI scripts.

Dataset link:

https://www.kaggle.com/code/vishantmathur/employee-salary

Expected Output:

- A structured dataset ready for HR analytics.

Prompt:

Generate a Python pandas script to preprocess an employee salary dataset by handling missing values, detecting and removing salary outliers,standardizing department names to lowercase, encoding categorical variables (job location, department), and outputting a cleaned dataset ready for HR analytics.

Code:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
import numpy as np

# Load dataset
try:
    data = pd.read_csv("/content/employee_salary.csv")
except FileNotFoundError:
    print("Error: employee_salary.csv not found. Please ensure the file is in the correct directory.")
    exit()

print("Original Data Head:")
print(data.head())
print("\nMissing Values before preprocessing:")
print(data.isnull().sum())

# 1. Handle Missing Values
# Assuming 'Salary' is numerical, fill with median
if 'Salary' in data.columns:
    data['Salary'].fillna(data['Salary'].median(), inplace=True)
# Assuming 'Department' and 'Job Location' are categorical, fill with mode
if 'Department' in data.columns:
    data['Department'].fillna(data['Department'].mode()[0], inplace=True)
if 'Job Location' in data.columns:
    data['Job Location'].fillna(data['Job Location'].mode()[0], inplace=True)

print("\nMissing Values after handling:")
print(data.isnull().sum())
```

```python
# 2. Detect & Remove Salary Outliers (IQR method)
if 'Salary' in data.columns:
    Q1 = data['Salary'].quantile(0.25)
    Q3 = data['Salary'].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    initial_rows = data.shape[0]
    data = data[(data['Salary'] >= lower_bound) & (data['Salary'] <= upper_bound)]
    removed_rows = initial_rows - data.shape[0]
    print(f"\nRemoved {removed_rows} salary outliers.")
else:
    print("\n'Salary' column not found, skipping outlier detection.")

# 3. Standardize Department Names to lowercase
if 'Department' in data.columns:
    data['Department'] = data['Department'].str.lower().str.strip()
    print("Standardized 'Department' names to lowercase.")
else:
    print("\n'Department' column not found, skipping standardization.")

# 4. Encode Categorical Variables
encoder = LabelEncoder()
categorical_cols_to_encode = []
```

```python
# 4. Encode Categorical Variables
encoder = LabelEncoder()
categorical_cols_to_encode = []

if 'Job Location' in data.columns:
    categorical_cols_to_encode.append('Job Location')
if 'Department' in data.columns:
    # Only encode if not already numerical from previous steps (though not expected here)
    if data['Department'].dtype == 'object': # Check if it's still an object type
        categorical_cols_to_encode.append('Department')

for col in categorical_cols_to_encode:
    data[col] = encoder.fit_transform(data[col])
    print(f"Encoded column '{col}' with LabelEncoder.")

if not categorical_cols_to_encode:
    print("\nNo 'Job Location' or 'Department' columns found for encoding.")

# Final Output
print("\nProcessed Data Head (ready for HR analytics):")
print(data.head())

# Save cleaned dataset
data.to_csv("cleaned_employee_salary.csv", index=False)
```

Output:

```
Original Data Head:
                                          code_cells
0  import numpy as np\nimport pandas as pd\nimpor...
1  df1 = pd.read_csv("/kaggle/input/employees-dep...
2                     df = pd.merge(df1,df2,on="ID")
3                                          df.head()
4                                           df.shape

Missing Values before preprocessing:
code_cells    0
dtype: int64

Missing Values after handling:
code_cells    0
dtype: int64

'Salary' column not found, skipping outlier detection.

'Department' column not found, skipping standardization.

No 'Job Location' or 'Department' columns found for encoding.

Processed Data Head (ready for HR analytics):
                                          code_cells
0  import numpy as np\nimport pandas as pd\nimpor...
1  df1 = pd.read_csv("/kaggle/input/employees-dep...
2                     df = pd.merge(df1,df2,on="ID")
3                                          df.head()
4                                           df.shape

Cleaned data saved to 'cleaned_employee_salary.csv'
```

Justification:

Preprocessing employee salary data improves data quality by handling missing values and removing inconsistencies.
Outlier detection ensures unrealistic salary values do not affect analysis results.
Standardizing and encoding features makes the dataset suitable for HR analytics and machine learning models.